Clase 35 — Fragmentación, MTU y Path MTU

Bloque: D (Capa de Red) · **Semana:** 12 · **Duración estimada:** 120 min **Correlativa:** Clase 34 (BGP y el routing entre operadores) · **Sigue con:** Clase 36 (IPv6) **Material de referencia:** Tanenbaum cap. 5 (§5.5.2) · RFC 791, RFC 1122, RFC 1191, RFC 8200 §5, RFC 8899

🎯 Al terminar esta clase vas a poder

1. Definir **MTU**, **fragmentación** y **Path MTU**, y explicar por qué el Path MTU es el **mínimo** de los MTUs del camino.
2. Dibujar la cabecera IPv4 señalando los **tres campos** de la fragmentación: `Identification`, `Flags` (con `DF`) y `Fragment Offset`.
3. **Escribir el comportamiento de un router** frente a un paquete que no entra, y justificar por qué **descarta** cuando `DF = 1`.
4. Enumerar los **problemas asociados**: unidad de 8 bytes, overhead, temporizador, solapamiento, pérdida de un fragmento, evasión de filtros.
5. **Implementar el algoritmo de reensamblaje** con índice por offset, detección de solapamiento y temporizador.
6. **Implementar el descubrimiento del Path MTU** con `enviar_msg_ICMP` / `recibir_msg_ICMP`, y explicar por qué la solución oficial subestima.
7. Explicar qué es el **PMTUD black hole** y diagnosticarlo con `ping -M do`.

> **Prerequisito del examen:** `2010-12 Problema 2` (30 pts — MTU, fragmentación, Path MTU **e implementación del descubrimiento**), `2006-12 P3` (10 pts — **algoritmo de reensamblaje**), `2000-07 Problema 2`.
>
> ⚠️ Es **una de las clases más importantes del curso**: el problema de 2010-12 vale 30 puntos y su último apartado es un algoritmo a implementar.

PARTE 1 — TEORÍA

1.1 MTU: la unidad máxima de transmisión

MTU (*Maximum Transmission Unit*) es **la mayor PDU (unidad de datos de protocolo) de capa 3 que un enlace transmite sin fragmentar**.

Existe porque la física lo impone: a velocidad `R`, transmitir `L` bytes lleva `L/R` segundos, así que un datagrama de 64 KB monopolizaría el enlace medio segundo; además la memoria de la interfaz y el tamaño máximo de trama del hardware son finitos.

Ojo con una fuente típica de errores: la MTU cuenta solo los datos, no la cabecera de capa 2.

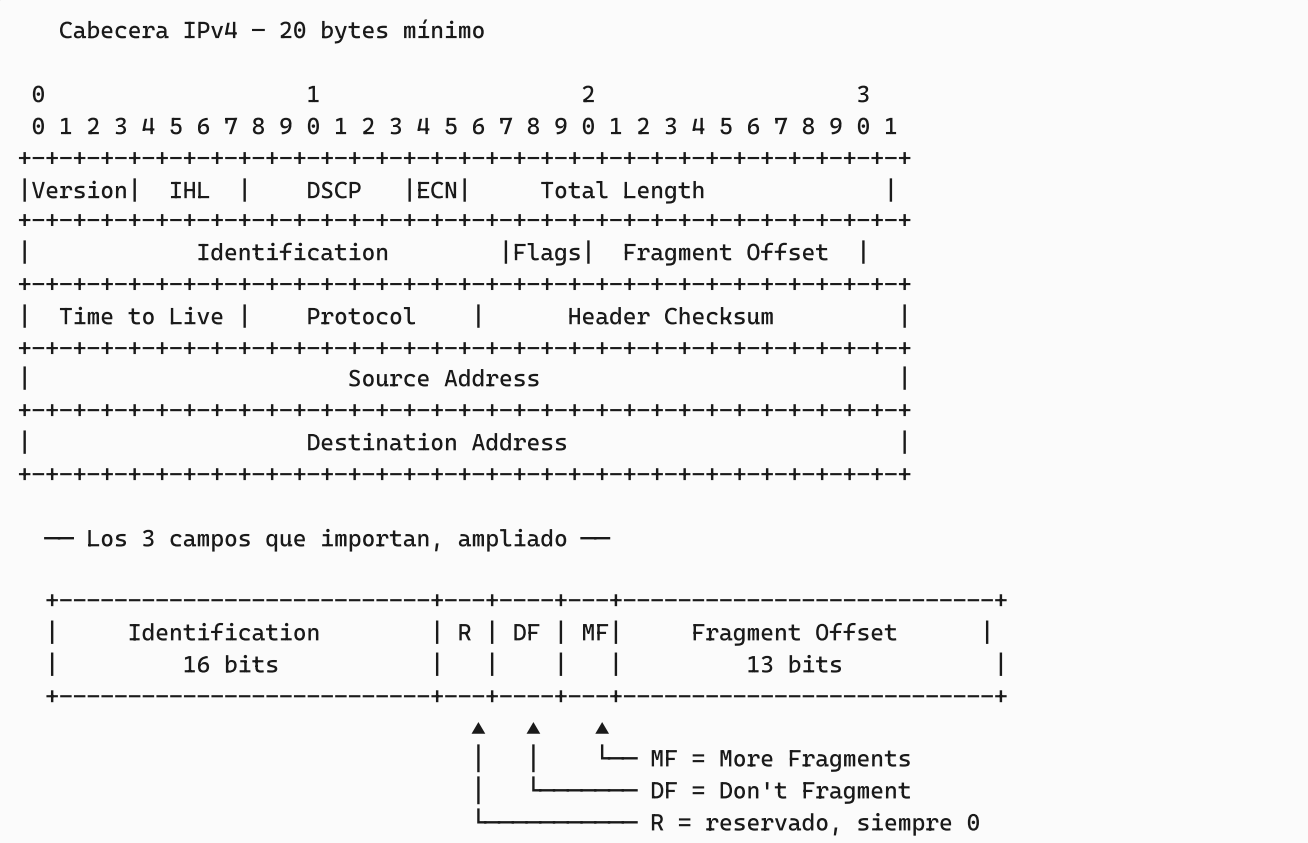
```
Trama Ethernet máxima:  1518 bytes
   ├─ MAC destino 6 + MAC origen 6 + Ethertype 2 = 14
   ├─ DATOS IP       1500   ◄── ESTO es la MTU
   └─ FCS              4
──────────────────────────────────  14 + 1500 + 4 = 1518
```

Enlace	MTU	Origen del número
Ethernet	**1500**	valor por defecto de IEEE 802.3
Jumbo frame	**9000**	**convención, NO estándar IEEE**
PPPoE (ADSL)	**1492**	1500 menos 8 de encapsulación
Frame Relay / ATM	Variable	según VC; típico 1500
Token Ring 16 Mbps	17918	el estándar lo fija explícitamente
IPv4 mínimo absoluto	**68**	`RFC 791`; en la práctica se/supporta 576
IPv6 mínimo absoluto	**1280**	`RFC 8200 §5`, verificado textualmente

> ⚠️ El **jumbo de 9000 es una convención de la industria, no un estándar IEEE**. Se usa intra-datacenter y en almacenamiento. **Nunca sobre Internet ni sobre túneles sin *clamp MSS***: si un router del camino tiene MTU 1500 y vos mandás 9000, o fragmentás (IPv4) o perdés la conexión (IPv6, que no fragmenta).

El **68 bytes** es el datagrama mínimo absoluto de `RFC 791` (60 de cabecera con opciones + 8 de datos), pero `RFC 1122 §3.2.1` pide que **todo host soporte al menos 576 bytes**. El 68 es un piso histórico; nadie baja de 576 en una red real. El mínimo de IPv6, en cambio, es **1280 y obligatorio para toda interfaz**, sin excepción por opciones.

1.2 Fragmentación: los tres campos que la habilitan



Campo	Bits	Función
Identification	16	El **"número de pedido"** del datagrama. **Todos los fragmentos llevan el mismo ID.** El destino agrupa por `(IP destino, ID)`.
Flags: DF	1	**Decide si se puede fragmentar.** `0` = se puede; `1` = prohibido: si no entra, se descarta y se avisa.
Flags: MF	1	`1` = hay más fragmentos; `0` = **este es el último**.
Fragment Offset	13	Posición del payload de este fragmento dentro del payload original, **en unidades de 8 bytes**.

Por qué 8 bytes y no 1: economía de bits. 13 bits en unidades de bytes darían solo 8192 bytes de payload máximo. En unidades de 8 dan 8191 × 8 = **65528 bytes**, que cubre cualquier datagrama IPv4 válido. La consecuencia es dura: **el payload de todo fragmento salvo el último debe ser múltiplo de 8**, así que si el router tiene 556 bytes disponibles, recorta a 552 y deja 4 para el siguiente.

Ejemplo. Datagrama de 1500 bytes (`IHL = 5`, payload 1480) por un enlace de MTU 576:

```
Original:  [cabecera 20 | payload 1480]                    longitud = 1500

F1: cabecera 20 | payload 552                             longitud =  572
    offset = 0/8 = 0          MF = 1
F2: cabecera 20 | payload 552                             longitud =  572
    offset = 552/8 = 69       MF = 1
F3: cabecera 20 | payload 1480-1104 = 376                 longitud =  396
    offset = 1104/8 = 138     MF = 0
_____________________________________________________________________
Sobre el enlace: 572 + 572 + 396 = 1540   →  overhead 40 bytes (2,7 %)
Todos con el MISMO Identification, mismo src y mismo dst.
```

Reglas de memoria: el `Fragment Offset` va en unidades de 8; el `ID` se copia tal cual; el **TTL se restaura al valor original** (los fragmentos viajan de a uno, no en paralelo); el **checksum de cabecera se recalcula** porque la cabecera cambió; el **checksum de capa 4 no cambia** (se verifica recién al reensamblar); y **cada fragmento es un datagrama IP independiente**, así que puede tomar otro camino si hay ECMP.

1.3 El comportamiento del router — lo que pide el Examen 2010-12

Este pseudocode es **literalmente** el apartado (b) del problema de 30 puntos:

```
// ---- Comportamiento de un router al reenviar por un enlace de MTU MTUo ----
//   Pi    = paquete entrante
//   L     = longitud total de Pi (campo Total Length)
//   MTUo  = MTU del enlace saliente

Pi = paquete entrante ;  L = Pi.Total_Length

if (L <= MTUo)                      →  // CASO 1: cabe entero
    transmitir Pi normalmente                       // sin importar DF

else if (L > MTUo and Pi.DF == 1)   →  // CASO 2: no cabe y NO se puede partir
    DESCARTA Pi
    envía ICMP (destino = Pi.origen, tipo = 3, código = 4)
    //   tipo 3 = Destination Unreachable
    //   código 4 = "Fragmentation Needed and Don't Fragment was Set"

else if (L > MTUo and Pi.DF == 0)   →  // CASO 3: no cabe y se PUEDE partir
    FRAGMENTAR Pi
    transmitir cada fragmento
```

Por qué el orden importa: se chequea primero si cabe, porque es el caso más frecuente y el más barato (ni hace falta mirar los flags).

¿Por qué descarta con `DF = 1` en vez de fragmentar igual? Porque `DF` significa "no me fragmentes": el emisor **declaró un límite que hay que respetar**, y un router que lo fragmentara produciría algo que el receptor final **no sabe tratar** (una aplicación entrega datos a un socket, no a una bolsa de fragmentos).

¿Por qué manda el ICMP? Tres razones, y las dos primeras son los puntos:

- **Para que el emisor no repita el error.** IP es *best effort*: si nadie avisa, el emisor solo ve pérdida y **retransmite el mismo datagrama idéntico**, que vuelve a ser descartado. Bucle infinito.

- **Para que el emisor se adapte.** El tipo 3 código 4 **lleva la MTU exacta** del enlace bloqueante en los **bytes 6-7** (`RFC 1191`), así que puede bajar su MSS o su caché de PMTU.
- **Para que la red sea observable.** Sin reportes, el operador nunca se enteraría de que su MTU está mal.

⚠️ **Riesgo:** un atacante que inyecte ICMP tipo 3 código 4 falsos puede degradar el servicio. Por eso los routers hoy **no emiten ICMP hacia multicast** y muchos limitan la tasa de errores que generan.

1.4 El problema de la fragmentación

Este es el corazón conceptual del `2006-12 P3`. La fragmentación se paga cara:

(a) Unidad mínima de 8 bytes. Implica el recorte descrito en §1.2. Si un enlace deja pasar solo 8 bytes de payload por fragmento, se generan muchísimos fragmentos.

(b) Overhead de cada cabecera. Cada fragmento repite los 20 bytes de cabecera IP:

```
Payload real = 1500 − 20 = 1480 bytes.  Si entran 8 bytes de payload:
    n = 1480 / 8 = 185 fragmentos
    Sobre el enlace: 185 × 28 = 5180 bytes
    Overhead: 5180 − 1500 = 3680 bytes  →  el datagrama CRECIÓ un 245 %
```

> ⚠️ La cifra de **"188 fragmentos"** que circula en los apuntes sale de `1500/8 = 187,5`. **El cálculo correcto es sobre el payload (1480), no sobre la longitud total**, porque la cabecera se reescribe en cada fragmento. Si el examen te da la longitud total, **descontá `IHL × 4` primero**.

(c) Reensamblaje con temporizador. Un fragmento que llega solo no se puede entregar, así que el destino **guarda estado** y arranca un temporizador (típicamente 60–120 s) al recibir el primero. Si vence sin completarse, **descarta todos los fragmentos y libera la memoria** — y el datagrama **se pierde entero**.

(d) Solapamiento de fragmentos — el agujero de seguridad. Dos fragmentos se solapan si cubren bytes del payload original en común. `RFC 791` dice que ante solapamiento se descarte el datagrama, pero **la implementación real, históricamente, no siempre lo hizo**. El problema es que **el segundo fragmento sobrescribe al primero**: un atacante que inyecte un fragmento con el mismo `(IP destino, ID, offset)` pero datos distintos puede **cambiar el contenido del datagrama**. Es la base de los ataques clásicos de fragmentación (*fragment overlap*, *teardrop*). IPv6 lo cierra: `RFC 5722` prohíbe explícitamente el solapamiento y `RFC 6946` recomienda tratar los **fragmentos atómicos** como no fragmentados.

(e) La pérdida de un fragmento descarta el datagrama entero. Y es multiplicativo:

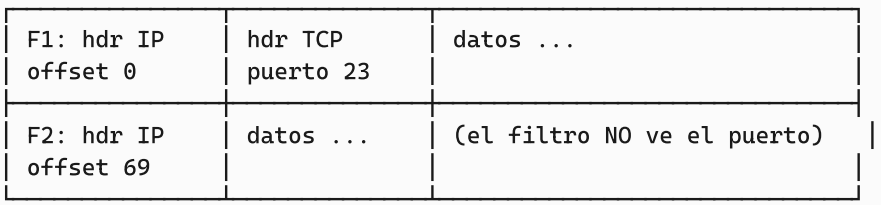
```
Si el enlace pierde con probabilidad p, y el datagrama se partió en n:
    P(que el datagrama ENTERO llegue) = (1 − p)^n

p = 0,01:   n = 3 → 0,99³  = 0,9703  →  97,0 % de éxito
            n = 185 → 0,99^185 = 0,157  →  15,7 % de éxito
```

Interpretación: fragmentar 185 veces multiplicó por 6 la probabilidad de pérdida del datagrama completo. **El datagrama fragmentado tiene una probabilidad de pérdida mucho mayor que uno que no lo está**, aunque el enlace individual sea idéntico.

(f) El ataque de fragmentación en capas — el punto de seguridad.

> **Cada fragmento es un paquete IP independiente para cada firewall o filtro que atraviesa.** Un filtro que aplica reglas sobre campos del datagrama **solo ve cada fragmento por separado**.



Un atacante **fragmenta el datagrama para esconder un puerto prohibido en un fragmento que el filtro no inspecciona**. Y hay una capa más: fragmentando, el primer fragmento lleva solo los 8 bytes obligatorios de payload, y **los datos del protocolo quedan en los fragmentos posteriores**, donde un filtro *stateful* que hace tracking de conexiones no los asoció. **Por eso los firewalls modernos reensamblan antes de decidir**, y por eso en IPv6 — donde el router nunca fragmenta — este ataque es estructuralmente mucho más difícil.

1.5 El algoritmo de reensamblaje

La solución oficial de `2006-12` es un esqueleto; esta es la versión completa. **Suposiciones que hago, justificadas** (el enunciado pide justificarlas):

1. **Los fragmentos pueden llegar en cualquier orden y más de una vez.** IP es *best effort* y no reordena.
2. **Se agrupan por el triple `(IP destino, Identification, Protocolo)`.** Es lo único que el destino tiene para saber a qué datagrama pertenece un fragmento.
3. **Hay que detectar solapamiento**, porque si no un fragmento puede sobrescribir bytes ya entregados.

```
// ======== ESTADO POR DATAGRAMA EN CURSO ========
TablaEnsamblaje[ (IP_destino, ID, protocolo) ] = {
    buffer[ fragment_offset ] , ocupado[ fragment_offset ] ,
    cabecera_original , longitud_total = -1 , TTL_assembly
}

AL_RECIBIR_DATAGRAMA( Pi ):
    E = TablaEnsamblaje[ (Pi.destino, Pi.Identification, Pi.protocolo) ]
    if (E no existe): crear E vacía ; E.TTL = ahora() + 120 s

    // (0) el fragmento de offset 0 lleva la CABECERA ORIGINAL (con sus
    //     opciones). Hay que guardarla aparte: el Total_Length de P no es
    //     el del datagrama original.
    if (Pi.Fragment_Offset == 0): E.cabecera_original = Pi

    // (1) longitud mínima del payload
    if (payload(Pi) < 8 and Pi.MF == 1): descartar Pi ; return

    // (2) SOLAPAMIENTO  →  política conservadora: se descarta todo
    inicio = Pi.Fragment_Offset × 8 ;  fin = inicio + payload(Pi)
    for cada offset o en E.ocupado:
        if (o ≠ Pi.Fragment_Offset) and los intervalos se cruzan:
            descartar Pi ; liberar E ; return

    // (3) almacenar EN SU POSICIÓN
    E.buffer[ Pi.Fragment_Offset ] = Pi ;  E.ocupado[ Pi.Fragment_Offset ] = true
    E.TTL = ahora() + 120 s

    // (4) si es el ÚLTIMO, ya sabemos el largo total del payload
    if (Pi.MF == 0):
        E.longitud_total = (Pi.Fragment_Offset × 8) + payload(Pi)

    // (5) ¿COMPLETO?  No por MF, por COBERTURA TOTAL
    if (E.longitud_total ≠ -1) and todos_los_bytes_cubiertos(E, E.longitud_total):
        Ptot = reensamblar(E)          // arma el datagrama original
        Ptot.TTL = Pi.TTL              // el menor TTL de los fragmentos
        entregar Ptot a la capa 3 ;  liberar E ;  return

// ======== BARRIDO PERIÓDICO (no corre por fragmento recibido) ========
for cada E en TablaEnsamblaje:
    if (ahora() > E.TTL):
        descartar todos los fragmentos de E      // el datagrama se pierde entero
        liberar E
```

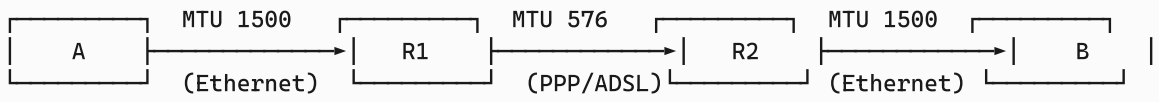
El concepto del paso 5, que es lo que separa "andar" de "entender": el datagrama **no está completo cuando llegó el último fragmento**, sino cuando **todos los bytes del intervalo `[0, longitud_total)` están cubiertos sin huecos**. Con 3 fragmentos que llegan en orden los dos hechos coinciden; con 5 que llegan desordenados **no**:

```
CASO en que "el último" NO implica "completo":
   llega F5 (el último, MF = 0, offset 2208, payload 424)
   estado real: hay un solo fragmento de 5 → falta el 80 % del datagrama
   Un algoritmo "reensamblo al ver MF = 0" entrega BASURA.

CASO en que sí coincide (por casualidad):
   F1 ✓  F3 ✓  F4 ✓  F2 ✓  F5 ✓  → llega F5 y en ese instante todo está
   cubierto. Funciona, pero solo por casualidad del orden.
```

1.6 Path MTU: el mínimo del camino

Path MTU es **la MTU del camino completo entre dos extremos: el mínimo de las MTUs de todos los enlaces del camino.**




```
Path MTU de A hacia B = min(1500, 576, 1500) = 576
⇐ El cuello de botella es el enlace del medio, NO los extremos.
```

Tres propiedades que hay que decir siempre, porque el enunciado de 2010-12 las pide explícitamente:

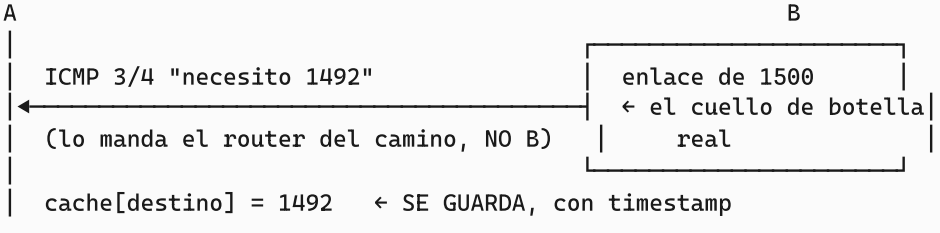
1. **Puede variar con el tiempo** si **los caminos cambian**: con recálculo de rutas (OSPF, IS-IS) o con **ECMP** repartiendo carga. Un PMTU cacheado puede quedarse viejo.
2. **Puede ser distinto en cada sentido** si el **ruteo es asimétrico**. Hay dos caminos y dos PMTUs; es el caso normal en Internet.
3. **Es una propiedad del camino, no del prefijo destino.** Dos flujos al mismo destino pueden tener PMTU distinto si ECMP los manda por enlaces distintos.

La diferencia entre fragmentar y descubrir el PMTU, en una frase: la fragmentación **hace que el paquete llegue**; el descubrimiento **hace que el paquete nunca sea demasiado grande**. La primera gasta ancho de banda y probabilidad de éxito; el segundo no gasta nada.

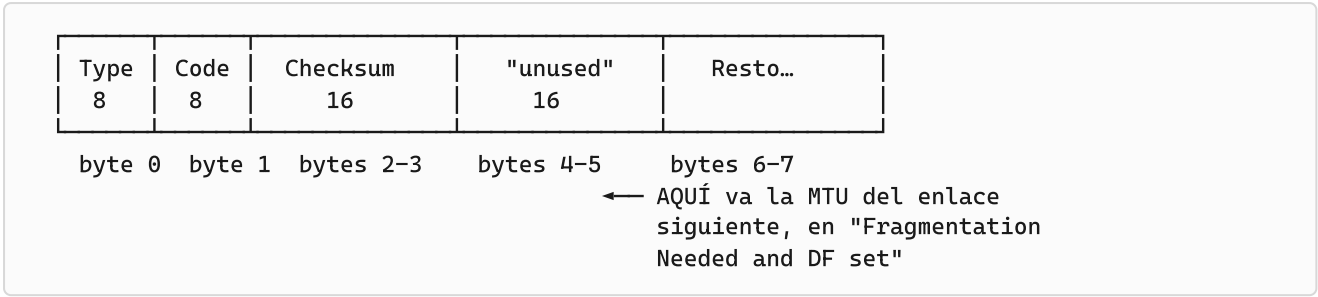
1.7 Path MTU Discovery (`RFC 1191`)

Tres pasos, y el tercero es el esencial:

```
1. El emisor pone DF = 1 en todos sus datagramas (TCP lo hace por omisión).
2. El router que no puede pasarlo:
       descarta el datagrama
       envía ICMP tipo 3 código 4 con la MTU del enlace en los bytes 6–7
3. El emisor recibe el ICMP, BAJA SU CACHÉ de PMTU para ese destino y
   vuelve a mandar con un tamaño que sí entra.
```



El campo de la MTU dentro del ICMP (cabecera de 8 bytes):



> `RFC 1191 §3` usa ese campo "unused" para llevar **la MTU del enlace siguiente** en el mensaje *Fragmentation Needed and DF Set*. **Es la respuesta directa al problema:** el router no dice solamente "no entra", dice **"entra hasta acá"**. En la práctica muchos routers no lo llenan, y por eso la solución oficial multiplica por 0.85 (§3.1c).

La degradación elegante — y su falla silenciosa: si el ICMP tipo 3 código 4 **no llega nunca** (un firewall lo filtró, o el router que debía emitirlo está roto), el emisor **no se entera de nada**. El paquete se pierde, se retransmite, se pierde otra vez, y la conexión se cuelga sin explicación. Ese es el **black hole**.

1.8 MTU y MSS: la defensa de TCP

La capa de transporte evita la fragmentación **por su cuenta**, anunciando su **MSS** (*Maximum Segment Size*) en el SYN:

```
MSS = MTU del enlace de SALIDA del emisor − cabeceras de IP y TCP

IPv4:  MSS = MTU − 20 (IP) − 20 (TCP) = MTU − 40   →  1460 en Ethernet
IPv6:  MSS = MTU − 40 (IP) − 20 (TCP) = MTU − 60   →  1420 en Ethernet
```

Dos extremos que conectan por caminos distintos usan **`min(MSS_A, MSS_B)`**, y con eso ningún segmento puede superar el PMTU. **Los segmentos TCP nunca se fragmentan**: si uno no entra en un enlace intermedio, **se descarta y se retransmite más chico**.

El clamp MSS. Con un jumbo frame (9000) en un borde y un lado que no lo soporta, alguien tiene que **recortar el MSS**:

```
Servidor en red de jumbo:  MSS anunciado = 8960
Cliente detrás de un enlace de 1500:  no puede recibir segmentos de 8960
⇐ El firewall / router de borde hace CLAMP:  MSS = min(8960, 1500−40) = 1460
⇐ Por eso "jumbo frames nunca sin clamp MSS": sin clamp, se rompe.
```

1.9 El PMTUD black hole

Qué es: un router que no puede pasar el datagrama **no genera el ICMP** (porque un firewall lo filtró, porque un ACL no lo contempla, o porque el router está roto). El emisor nunca se entera de que su PMTU cacheado está mal, sigue mandando datagramas de tamaño fijo, y **todos se pierden en silencio**.

El síntoma clásico, de memoria:

> **"SSH funciona y HTTPS se queda colgado"** (o al revés, según qué protocolo sobreviva).

¿Por qué un protocolo funciona y el otro no? Porque **no todos los datagramas tienen el mismo tamaño**:

```
SSH interactivo: paquetes chicos (prompt, teclas), ~100 bytes.
                 100 < 1280  ⇐  pasan por el cuello de botella.
                 ⇐ SSH FUNCIONA

HTTPS: el servidor manda respuestas del tamaño del MSS negociado (1460).
       1460 > 1280  ⇐  el router del cuello lo descarta, sin avisar.
       ⇐ HTTPS SE CUELGA
```

Diagnóstico:

```
# Linux: -M do = forzar DF=1 (don't fragment); -s = tamaño del payload
ping -M do -s 1400 ejemplo.com      # falla  → hay un cuello por debajo de 1400
ping -M do -s 1000 ejemplo.com      # funciona → el cuello está entre 1000 y 1400
tracepath ejemplo.com               # muestra el PMTU de cada salto
# Windows:  ping -f -l 1400 ejemplo.com      (-f = DF, -l = payload)
```

Soluciones, de mejor a peor: (1) **dejar pasar el ICMP tipo 3 código 4** en los firewalls — es lo correcto; (2) **DPLPMTUD** (`RFC 8899`), que **no depende de ICMP**; (3) **MSS clamping** en el borde — muy desplegado; (4) **caché con TTL corto y revalidación**; (5) bajar el MSS del socket a mano — último recurso.

1.10 Actualización a 2026

Sigue igual, de memoria:

- **MTU** = la mayor PDU de capa 3 que el enlace transmite sin fragmentar.
- **Ethernet 1500.** Jumbo 9000.
- **IPv4 fragmenta en los routers (DF = 0); IPv6 fragmenta solo en el origen.**
- **ICMPv4 tipo 3 código 4** vs. **ICMPv6 tipo 2** (*Packet Too Big*).
- **El PMTU se cachea en el origen**, es **por ruta**, y es **≤ al MTU del salto más estrecho**.
- El **"576 mínimo"** en IPv4 viene de `RFC 791` y `RFC 1122 §3.2.1`.
- **IPv6 mínimo absoluto = 1280** (`RFC 8200 §5`).

Cambió:

- `RFC 4821` **(2007, *Experimental*) →** `RFC 8899` ***(sep-2020, Standards Track): DPLPMTUD (Datagram PLPMTUD). La PMTUD dejó de depender de ICMP: el transporte sondea con su propio paquete de datos. Es la respuesta canónica al black hole y es lo que QUIC usa por defecto (*** `RFC 9002 §6` ***).*** ⚠️ ***DPLPMTUD es loss-based***, lo que introduce ambigüedad entre pérdida por MTU y pérdida por congestión; pocas implementaciones lo soportan (en Linux es una opción, no el default del stack TCP).

- **El "9000 → 1500 cliff"** es hoy un problema real: con **ECMP**, el PMTU es **propiedad del camino, no del prefijo**, así que flujos distintos al mismo destino pueden tener PMTU distinto.
- **Cada overlay consume MTU**: MPLS, VXLAN, IPsec, GRE y WireGuard. Un VXLAN sobre 1500 tiene MTU efectiva **1450**; con WireGuard sobre 1500, **1420**. Por eso el desajuste de MTU es **omnipresente** en redes cloud/bare-metal híbridas.
- **Hardening de fragmentos IPv6**: `RFC 5722` prohíbe solapamientos; `RFC 6946` trata los fragmentos atómicos como no fragmentados; `RFC 5095` deprecó el Routing Header tipo 0.
- **Síntoma clásico del black hole**: "SSH funciona, algunos sitios HTTPS quedan colgados". Diagnóstico: `ping -M do -s <size>`, `tracepath`.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — ¿Cuántos fragmentos genera un datagrama de 1500 bytes por un enlace de MTU 500?

Datos. Datagrama IP de **1500 bytes**, `IHL = 5` (cabecera de 20, sin opciones), `ID = 0xABCD`, `DF = 0`. Enlace de salida con **MTU = 500**.

Paso 1 — Descontar la cabecera del original.

```
Longitud total = 1500 ;  cabecera = IHL × 4 = 5 × 4 = 20
PAYLOAD REAL = 1500 − 20 = 1480 bytes      ⚠️ 1480, NO 1500
```

Paso 2 — Payload por fragmento. Cada fragmento tiene su propia cabecera de 20:

```
payload por fragmento = 500 − 20 = 480 ;  chequeo de 8 bytes: 480/8 = 60  ✓
```

Paso 3 — Cantidad de fragmentos.

```
n = ceil(1480 / 480) = ceil(3,083) = 4 fragmentos
```

#	Offset (bytes)	Offset en unidades de 8	Payload	Total Length	MF	ID
F1	0	0/8 = 0	480	500	1	0xABCD
F2	480	480/8 = 60	480	500	1	0xABCD
F3	960	960/8 = 120	480	500	1	0xABCD
F4	1440	1440/8 = 180	40	60	0	0xABCD

Verificación: `480+480+480+40 = 1480` ✓ igual al payload original. F4 tiene `1480 − 1440 = 40` bytes.

¿Cuántos bytes viajan?

```
500 + 500 + 500 + 60 = 1560 bytes
Overhead = 1560 − 1500 = 60 bytes  =  60/1500 = 4,0 %
```

Interpretación. Pagamos **60 bytes de overhead** (tres cabeceras repetidas) para que el datagrama pase de 1500 a 1560 bytes en el cable. Y el costo real es otro: **el receptor tiene que recibir 4 datagramas para entregar uno**, y si se pierde cualquiera de los cuatro se perdió el datagrama entero.

¿Y si viniera con `DF = 1`? `L = 1500 > 500` y `DF = 1` → **descarta** y manda **ICMP tipo 3 código 4** a `Pi.origen`. Cero bytes salen por ese enlace.

⚠️ Errores típicos

- **Usar 1500 como si fuera el payload.** Siempre `payload = longitud total − IHL × 4`.
- **Contar los offsets en bytes** en vez de unidades de 8.

- **Poner `MF = 1` en el último fragmento.** `MF = 0` es lo que le dice al receptor "acá termina, y con esto sé el largo total".

2.2 Ejercicio 2 — Reensamblar 4 fragmentos que llegan desordenados

Datos. Los 4 fragmentos de 2.1, pero llegan en el orden **F3, F1, F4, F2**. Todos con `ID = 0xABCD`.

Llegada	Offset	Payload	MF	Bytes cubiertos	Falta	¿Completo?
F3	960	480	1	960..1439	0..959 y 1440..1479	no
F1	0	480	1	0..479, 960..1439	480..959 y 1440..1479	no
F4	1440	40	**0**	0..479, 960..1479	**480..959**	**no**
F2	480	480	1	**0..1479**	—	**¡sí!**

Paso a paso, con la aritmética:

```
── Llega F3 (offset 960, 480 bytes) ──────────────────────────────────
   Cubre 960 .. 960+480-1 = 960..1439
   MF = 1  →  NO sabemos todavía el largo total
   Tabla: [0]=libre  [60]=libre  [120]=F3  [180]=libre

── Llega F1 (offset 0, 480 bytes) ────────────────────────────────────
   Cubre 0 .. 479
   MF = 1  →  todavía no
   ⚠️ F1 tiene offset 0 ⇒ es el que CONTIENE LA CABECERA ORIGINAL
      (con sus opciones). Su Total_Length NO es el del datagrama original.
   Tabla: [0]=F1  [60]=libre  [120]=F3  [180]=libre

── Llega F4 (offset 1440, 40 bytes) ──────────────────────────────────
   Cubre 1440 .. 1479
   MF = 0  →  ¡ES EL ÚLTIMO!  Ahora sí sé el largo del payload:
        longitud_total = 1440 + 40 = 1480
   Tabla: [0]=F1  [60]=libre  [120]=F3  [180]=F4

   ⇐ ¿Completo?  Ocupados: 0, 120, 180.  Falta el 60 (bytes 480..959).
      Faltan 480 bytes de 1480  ⇐  NO ESTÁ COMPLETO  ⇐  no se entrega nada.

── Llega F2 (offset 480, 480 bytes) ──────────────────────────────────
   Cubre 480 .. 959
   Tabla: [0]=F1  [60]=F2  [120]=F3  [180]=F4

   Verificación: cubiertos = 480+480+480+40 = 1480 ;  largo conocido = 1480
                 y los intervalos son contiguos y sin huecos  ⇐  ¡COMPLETO!
   ⇐ REENSAMBLO y entrega a la capa 3.
```

El datagrama reensamblado: cabecera tomada de **F1** (offset 0), con `IHL = 5` y `Total_Length = 20 + 1480 = 1500`; `Identification = 0xABCD`; `Flags`: `MF = 0`, `Fragment Offset = 0`; `TTL` = el menor de los fragmentos; checksum de cabecera recalculado. Payload = F1 (0..479) + F2 (480..959) + F3 (960..1439) + F4 (1440..1479). Recién **entonces** se verifica el checksum de TCP. **1500 bytes exactos, idéntico al original.**

¿Qué hubiera pasado con el algoritmo ingenuo? Uno que reensambla al ver `MF = 0` habría entregado un datagrama de **520 bytes** (F1 + F4, con un hueco de 480 bytes en el medio) cuando llegó F4. La capa de

transporte habría encontrado el checksum de TCP incorrecto y lo habría descartado. **Ese es justo el error que la solución oficial de `2006-12` evita con la suposición "los fragmentos llegaron en orden"**: es válida solo en ese caso.

⚠️ Errores típicos

- **Confundir `MF = 0` con "llegó todo".** Hay que contar los bytes cubiertos.
- **Olvidar que la cabecera original la tiene el fragmento de offset 0** (y solo él).
- **No reiniciar el temporizador** con cada fragmento: si el reensamblaje tarda 70 s y el temporizador es de 60, se descarta un datagrama que iba a completarse.

2.3 Ejercicio 3 — PMTUD con búsqueda binaria

Datos. MTU de la interfaz de A hacia B = **1500**. PMTU real = **1492** (hay un enlace PPPoE en el medio).

La versión oficial multiplica por 0.85. Ejecución con estos datos:

```
Pmtu = 1500 ; envío Echo Request de 1500 con DF = 1
  ⇐ el router del PPPoE lo descarta y manda ICMP 3/4
  ⇐ 1500 × 0,85 = 1275
envío de 1275 → llega entero → Echo Reply → return 1275
```

Devolvió 1275 en vez de 1492: 217 bytes por debajo = 14,5 % del ancho de banda desperdiciado. Y no es casualidad: los valores que genera son `1500 · 0,85^k`, y **ninguno de esos es 1492**. La solución oficial **subestima siempre**, como su propio comentario admite. El **pseudocode de mi versión y su desarrollo punto por punto están en §3.1c**; acá va la ejecución, que es lo que hay que saber hacer:

```
  Estado inicial:  lo = 68,  hi = 1500     (lo = mínimo legal, hi = MTU local)

  Probe 1:  mid = (68+1500)/2   =  784   → llega entero → lo =  784
  Probe 2:  mid = (784+1500)/2  = 1142   → llega entero → lo = 1142
  Probe 3:  mid = (1142+1500)/2 = 1321   → llega entero → lo = 1321
  Probe 4:  mid = (1321+1500)/2 = 1410   → llega entero → lo = 1410
  Probe 5:  mid = (1410+1500)/2 = 1455   → llega entero → lo = 1455
  Probe 6:  mid = (1455+1500)/2 = 1477   → llega entero → lo = 1477
  Probe 7:  mid = (1477+1500)/2 = 1488   → llega entero → lo = 1488
  Probe 8:  mid = (1488+1500)/2 = 1494   → 1494 > 1492  ⇐  ICMP 3/4 con
         MTU = 1492 en los bytes 6-7  ⇐  lo = hi = 1492  ⇐  termina
 ═══════════════════════════════════════════════════
  RESULTADO:  PMTU = 1492      ✓ EXACTO     (8 probes)
 ═══════════════════════════════════════════════════
```

	Oficial (`× 0.85`)	Mía (binaria + lee la MTU)
Resultado con estos datos	**1275**	**1492**
¿Exacto?	No, **siempre subestima**	**Sí**
Ancho de banda desperdiciado	217 B = **14,5 %**	**0**
Peor caso	~21 probes, y sigue mal	`log₂(1432)` = **11 probes**, exacto
¿Usa la MTU del ICMP?	**No**	**Sí** (bytes 6-7)
¿Cachea el resultado?	No	**Sí, por destino**
¿Distingue ruta rota de MTU chico?	No	**Sí**

Lo que le falta a mi versión también, y hay que decirlo en el examen: la tabla de ruteo debe poder **incrementar** el PMTU, no solo bajarlo. Si el cuello de botella desaparece, el caché queda viejo para siempre. La forma correcta es un **re-sondeo periódico**: cuando vence el `timestamp` del caché, sondear con un tamaño apenas mayor que el valor cacheado; si llega, subirlo.

⚠️ Errores típicos

- **Confundir `pkt.code == 0` con éxito sin mirar `pkt.tipo`.** El tipo 3 **también** tiene código 0 (net unreachable): el algoritmo diría que funcionó cuando no.
- **Devolver sin cachear.** El descubrimiento es caro; sin caché, cada conexión lo repite.
- **Redondear mal.** `mid = (lo+hi)/2` con enteros y el invariante `hi − lo > 1` son lo que garantiza la terminación.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-12, Problema 2 (30 puntos) — MTU, fragmentación y Path MTU

> **Fuente:** Examen `eirc1012` / `sirc1012` — 17 de diciembre de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1012.pdf

Enunciado

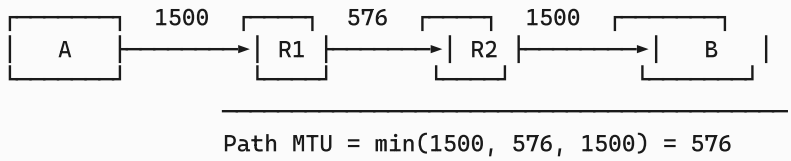
> El MTU (maximum transmission unit) es el tamaño máximo de paquete que puede pasar por determinado enlace. En base a esta definición y a lo visto en clase se pide:
>
> **a)** En una red de varios routers y enlaces, defina los conceptos de MTU, fragmentación y "Path MTU" (la MTU de un camino)
>
> **b)** Describa el comportamiento de un router al reenviar paquetes por un enlace con una MTU de "M" bytes. Describa los diferentes casos de acuerdo al valor del bit DF del encabezado IP, haciendo particular hincapié en los eventuales mensajes de control que se generen.
>
> **c)** Se desea implementar un algoritmo que permita determinar el "path MTU" entre dos nodos, A y B, conectados a la misma red IP. El algoritmo deberá estar basado en los mensajes ICMP que se pueden escuchar en la red y podrá enviar paquetes de prueba. El mismo deberá depender unicamente de uno de los extremos (es decir "A" no deberá requerir colaboración activa de "B", salvo el comportamiento por defecto del stack de protocolos). Se cuenta con las primitivas: `enviar_msg_ICMP(destino, tipo, código, largo_payload)` y `recibir_msg_ICMP(fuente)`. Esta última devuelve un mensaje ICMP recibido de la fuente o null en caso de que pase un cierto tiempo sin recibir nada (implementa un timeout).
>
> Se ofrece la siguiente tabla como referencia (tipos: **0** Echo Reply · **3** Destination Unreachable · **4** Source Quench · **5** Redirect · **8** Echo Request · **9** Router Advertisement · **10** Router Solicitation · **11** Time Exceeded · **12** Parameter Problem; códigos del tipo 3: **0** Net Unreachable · **1** Host Unreachable · **2** Protocol Unreachable · **3** Port Unreachable · **4** Fragmentation Needed and DF set · **5** Source Route Failed · **6** Destination Network Unknown · **7** Destination Host Unknown · **9**/10** comunicativamente prohibido).
>
> **Sobre la tabla:** el PDF la imprime en **dos columnas desalineadas** (la de nombres de tipos y la de códigos del tipo 3 quedaron desfasadas una posición). Leída correctamente es la tabla canónica de `RFC 792`. Los códigos que importan acá son el **3/4** ("fragmentation needed and DF set") y el **3/0** ("net unreachable", que **no** es un Echo Reply: es el bug de la solución oficial).

Parte a) — Las tres definiciones

MTU. Tamaño máximo de trama que una tecnología de capa 2 es capaz de transmitir; en la práctica, la mayor PDU de capa 3 que pasa sin fragmentarse. En Ethernet, 1500 bytes.

Fragmentación. Cuando un router debe enviar un paquete más largo que la MTU del enlace de salida, puede fragmentarlo y reenviarlo en partes. **Es una tarea pesada para un router, ya que le implica mantener estado de ciertos paquetes y sus fragmentos.**

Path MTU. Cuando dos hosts están separados por más de un hop, las MTUs de los distintos enlaces pueden diferir. Se define el **Path MTU como el mínimo de todos los MTUs involucrados en el camino** de un nodo a otro.



Y **las dos propiedades que la solución oficial exige mencionar**, porque están en el texto de la resolución y son parte de los puntos:

- **Puede variar con el tiempo** si los caminos cambian (ECMP o recálculo de rutas).
- **Puede ser diferente según el sentido del tráfico**: un mínimo para la ida y otro para la vuelta, **en el caso de que el ruteo sea asimétrico**.

Parte b) — El comportamiento del router

Solución oficial, literal:

```
DF: "Do not Fragment"

Paquete entrante Pi de largo L
MTU del enlace saliente MTUo

if L <= MTUo then
    transmitir normalmente Pi
else if L > MTUo and Pi.DF == 1 then
    descartar paquete Pi
    enviar msg ICMP (destino=Pi.source, tipo=3, codigo=4)
else if L > MTUo and Pi.DF == 0 then
    fragmentar paquete Pi
    transmitir fragmentos de Pi

Mensaje tipo 3: Destination unreachable
Codigo 4: "fragmentación necesaria pero DF encendido"
```

✅ **Lo que está bien:** el **orden de las condiciones** (primero "cabe", que es el caso frecuente y barato); la **distinción `DF=1` descarta vs. `DF=0` fragmenta**, que es el corazón de la pregunta; y **nombrar el tipo y el código** del ICMP.

🔶 **Lo que agregaría para completar los puntos:**

1. **Qué campos cambian al fragmentar**: `Total Length` se ajusta, `Fragment Offset` se setea, `MF = 1` en todos menos el último, y **el `Identification` se mantiene igual**. La pregunta pide "describa los diferentes casos", y describir *cómo* fragmenta es parte de la respuesta.
2. **La justificación del ICMP**, que la oficial da pero no explica: para que **el emisor no repita el error** (si no, retransmite el mismo datagrama idéntico en un bucle infinito) y para que **pueda reducir su tamaño** (el 3/4 trae la MTU del enlace bloqueante en los bytes 6-7).
3. **Que el último fragmento lleva `MF = 0`.**

Mi respuesta al apartado b), que es la que escribiría:

```
// ---- Router R, paquete entrante Pi, longitud L, MTU de salida MTUo ----
Pi.TTL = Pi.TTL - 1
if (Pi.TTL == 0):
    descartar Pi
    enviar ICMP (destino = Pi.origen, tipo = 11, código = 0)   // Time Exceeded
    return

if (L <= MTUo):                              // CASO 1: cabe entero
    recalcular Header_Checksum(Pi)
    transmitir Pi                             // sin importar DF
    return

if (Pi.DF == 1):                             // CASO 2: prohibido partir
    // El emisor declaró "Don't Fragment": el receptor final no sabe
    // tratar datos partidos y el paquete afirma un límite a respetar.
    descartar Pi
    enviar ICMP (destino = Pi.origen, tipo = 3, código = 4)
    //   tipo 3 = Destination Unreachable
    //   código 4 = "Fragmentation Needed and Don't Fragment was Set"
    //   los bytes 6-7 llevan la MTU del enlace bloqueante (RFC 1191),
    //   para que el emisor ajuste su MSS o su caché de PMTU.
    return

// CASO 3: no cabe y DF = 0  →  fragmentar
payload      = L - (Pi.IHL × 4)
max_payload  = ((MTUo - Pi.IHL × 4) / 8) × 8      // múltiplo de 8
offset = 0 ;  resto = payload
while (resto > 0):
    largo = min(max_payload, resto)
    F = copia de Pi
    F.Total_Length    = Pi.IHL × 4 + largo
    F.Identification  = Pi.Identification          // ← el MISMO en todos
    F.Fragment_Offset = offset / 8                 // ← en UNIDADES DE 8
    F.MF = (largo < resto) ? 1 : 0                 // 0 solo en el último
    recalcular Header_Checksum(F)
    transmitir F
    offset += largo ;  resto -= largo
```

> **Tope para los 10 puntos de b):** los tres casos con la condición exacta, el **descarte con `DF = 1`**, el **ICMP tipo 3 código 4 con su justificación**, y los campos que cambian al fragmentar.

Parte c) — El algoritmo de descubrimiento del Path MTU

Solución oficial, literal:

```
La estrategia básica es transmitir paquetes de prueba y escuchar los
mensajes ICMP de error que regresan. El algoritmo similar a una bipartición
pero simplificada.

Partimos de una resultado tentativo: sabemos que el path MTU no va a ser más
grande que el MTU del enlace por donde los paquetes destinados a B son
encaminados.

Pmtu = MTU(enlace hacia B)
tries = 0
while true
    // envio un ping hacia el destino de largo el maximo tentativo
    enviar_msg_ICMP (destino=B, tipo = 8, largo = Pmtu)
    pkt = recibir_msg_ICMP(fuente=B)
    if pkt != Null then
        if pkt.code == 0 then
            // es un echo reply, llego al destino bien
            return Pmtu
        else
            // calculo nueva aproximación
            // esta aproximación es bastante agresiva, se podría hacer mejor
            // tiende a sub-estimar el pmtu
            Pmtu = Pmtu * 0.85
    else
        // posiblemente timeout u otro error de red
        Pmtu = Pmtu * 0.85
        tries++
        if tries >= 5 then
            return False
```

✅ **Lo que está bien:** la estrategia (sondar con paquetes de prueba y escuchar los ICMP de error) es exactamente el PMTUD; **depende de un solo extremo** (A manda y espera, B no hace nada especial), cumpliendo la restricción del enunciado; usa **tipo 8** (Echo Request), que es lo que hace `ping`; **distingue el** `null` (timeout) de un mensaje recibido y cuenta reintentos; y **se autoevalúa**, reconociendo que la aproximación es agresiva.

⚠️ **Lo que está mal o es discutible — y esto también se pregunta:**

1. 🔴 **`if pkt.code == 0` sin mirar `pkt.tipo`.** El bug más grave: el **tipo 3 también tiene código 0** ("Net Unreachable"), como se ve en la propia tabla del examen. Si el router responde tipo 3 código 0, el algoritmo **devuelve un PMTU que nunca funcionó**. La condición correcta es `pkt.tipo == 0 AND pkt.codigo == 0`.
2. 🔴 **No usa la MTU que viene en el ICMP.** El `RFC 1191` la pone en los **bytes 6-7** del tipo 3 código 4. El enunciado da la tabla **justo para que uses ese mensaje**, y la respuesta **ya está dentro del ICMP**.
3. 🟡 **Nunca da el PMTU exacto.** Los valores son $1500 \cdot 0{,}85^k$; ninguno es 1492. Con PMTU real 1492 devuelve 1275 (**14,5 % bajo**).
4. 🟡 **El comentario no describe lo que hace el código.** Una bipartición es `(lo+hi)/2`; multiplicar por 0.85 es una búsqueda **exponencial hacia abajo**.
5. 🟡 **No cachea el resultado.** El *almacenamiento en caché* es parte del mecanismo: sin él, cada datagrama grande genera un ICMP y la red se llena de errores.
6. 🟡 **No hay cota inferior.** Puede bajar de 68 (el mínimo legal de IPv4) y seguir mandando sondas imposibles.

7. 🟡 **No distingue "MTU chico" de "ruta rota".** Un tipo 3 código 1 (host unreachable) o 9/10 (prohibido) no se arreglan bajando el tamaño.
8. 🟡 **Solo decrementa, nunca incrementa.** Si el camino mejora, el caché queda viejo para siempre.
9. 🔴 `tries` **solo cuenta en el timeout.** Si el router siempre responde tipo 3 código 4, el `while true` **no termina nunca**.

Mi solución: búsqueda binaria + lectura de la MTU del ICMP + caché.

```
// ======== ESTADO PERSISTENTE EN A ========
pm[B] = { valor, valido, timestamp }        // caché de PMTU por destino
MAX_REINTENTOS = 5 ;  MTU_MIN_ABSOLUTO = 68

int descubrir_PMTU(A, B):
    // ---- Invariantes ----
    //   lo : tamaño que YA sabemos que pasa entero  (cota inferior)
    //   hi : tamaño que YA sabemos que NO pasa      (cota superior)
    //   Al salir, lo es el mayor tamaño conocido como transmisible.
    if (pm[B].valido): lo = pm[B].valor      // arrancamos del caché
    else:              lo = MTU_MIN_ABSOLUTO // 68: seguro por definición
    hi = MTU(interfaz de A hacia B)           // 1500: no puede pasar más
    reintentos = 0

    while (hi - lo > 1):
        mid = (lo + hi) / 2                   // ← búsqueda BINARIA
        enviar_msg_ICMP(B, 8, 0, mid)         // Echo Request con DF = 1
        pkt = recibir_msg_ICMP(B)

        if (pkt == null):                      // timeout
            // Degradación elegante: asumimos el PEOR caso (que mid era
            // demasiado grande) y seguimos binary-searching.
            reintentos++
            if (reintentos > MAX_REINTENTOS): cachear(B, lo); return lo
            hi = mid

        else if (pkt.tipo == 0 and pkt.codigo == 0):
            lo = mid                          // llegó entero a B: ES válido

        else if (pkt.tipo == 3 and pkt.codigo == 4):
            // ★ AQUÍ ESTÁ LA MEJORA PRINCIPAL ★
            m = leer_uint16(pkt, offset = 6)  // Next-Hop MTU (RFC 1191)
            if (m >= lo and m < mid): lo = m; hi = m   // m ES transmisible
            else:                     hi = mid   // mensaje inconsistente

        else if (pkt.tipo == 3):
            // Otro inalcanzable (net, host, port, prohibido):
            // NO es un problema de tamaño, es un problema de ruta.
            cachear(B, lo); return ERROR_NO_EXISTE_RUTA

        else:                                  // tipo 11, 12, etc.
            reintentos++
            if (reintentos > MAX_REINTENTOS): cachear(B, lo); return lo
            hi = mid

    cachear(B, lo)
    return lo
```

Mejora	Por qué
`pkt.tipo == 0 && pkt.codigo == 0`	El tipo 3 tiene un código 0. Chequear solo el código hace que el algoritmo **diga que funcionó cuando no**. Bug real de la oficial.
Leo la MTU de los bytes 6-7	El mensaje **ya trae la respuesta exacta**. **1 solo probe** reemplaza a los 8 de la búsqueda binaria, y el resultado es **exacto** en vez de 14,5 % bajo.
Búsqueda binaria `(lo+hi)/2`	`O(log n)`: peor caso `log₂(1432)` = **11 probes**, **siempre**. El `× 0.85` nunca converge al valor exacto.
Caché por destino	El PMTU es **propiedad de la ruta**. Sin caché, cada paquete grande vuelve a generar un ICMP.
Cota inferior `lo = 68`	Nunca sondea algo que **no puede funcionar**.
Distinguir los otros tipo 3	Un "host unreachable" no se arregla bajando el MTU.
Contar todos los reintentos	La oficial puede no terminar nunca.

> **Nota sobre la diferencia con la oficial:** la oficial **funciona** como heurística y devuelve un valor **conservador**, lo cual es "correcto pero ineficiente". La mía devuelve el valor **exacto**. En el examen, **presentá las dos**: reproducí la oficial con su análisis, y después proponé la mejora con el argumento. Eso vale más puntos que una sola.
>
> **Tope para los 10 puntos de c):** pseudocode completo y ejecutable, la **inicialización del estado** (el `Pmtu` tentativo desde el MTU del enlace local), el **manejo del timeout**, la **lectura del tipo y el código** del ICMP recibido, y **al menos una justificación** de por qué tu estrategia es mejor que "probar y ver qué pasa".

3.2 Examen 2006-12, Pregunta 3 (10 puntos) — fragmentación y reensamblaje

> **Fuente:** Examen `sirc0612` — 26 de diciembre de 2006.
>
> **Enunciado:** Explique por qué puede ser necesaria la fragmentación de paquetes a nivel de capa 3. Explique cómo funciona la misma en IPv4 y mencione los problemas asociados a la misma. Dé un algoritmo para realizar el reensamblaje de un paquete IP a partir de sus fragmentos.

La respuesta oficial, completa:

> *a)* La fragmentación supone dividir un paquete en varios más pequeños. La necesidad se basa en que **todo medio de transmisión tiene un Maximum Transfer Unit**. Si un paquete es mayor que el MTU, debe fragmentarse.
>
> *b)* En cada fragmento: el **total length** se ajusta al tamaño del segmento; la bandera **MF** se coloca en uno excepto para el último; y se setea el **fragment offset** de acuerdo al offset del mismo en el segmento original.

> *c) El algoritmo de reensamblaje:* "El algorítmico básico sería el siguiente, en la solución propuesta **se supone que los fragmento llegaron en orden y el último llega con el bit MF en 0**": `while (true) { recibo_paquete(p); if ( p.MF == 1 ){ almaceno(p); } else { if (p.fragment_offset <> 0 ){ almaceno(p); p_tot = reensamblo(p); if (p_tot <> NULL){ envo_paquete(p_tot); } } } }`

Análisis: por qué es insuficiente, y qué corregir.

#	Problema	Por qué importa
1	**Asume que los fragmentos llegan en orden**	En una red real pueden llegar desordenados. De eso depende **todo** el algoritmo.
2	**`p_tot = reensamblo(p)` al ver `MF = 0`**	Solo funciona con la suposición 1. Ver §2.2 para el caso donde falla.
3	**`almaceno(p)` no dice dónde**	Sin índice por `Fragment Offset` no se puede saber qué falta.
4	**No hay detección de solapamiento**	Es el agujero de seguridad (§1.4 d).
5	**No hay temporizador**	Los fragmentos se acumulan para siempre → fuga de memoria del destino.
6	**No hay clave de agrupación**	Falta `(IP destino, Identification, protocolo)`.
7	**`if (p.fragment_offset <> 0)` descarta mal**	Si el último en llegar es el de offset 0, **se pierde**.

El algoritmo corregido con las 7 correcciones está desarrollado en **§1.5**: índice por `Fragment Offset`, clave de agrupación, chequeo de longitud mínima, detección de solapamiento con liberación del reensamblaje, `longitud_total` derivado del `MF = 0`, **completitud por cobertura total** (no por `MF`), `temporizador` reiniciado en cada fragmento y con barrido periódico.

Respuesta a "los problemas asociados", que es donde están la mitad de los puntos: (i) **overhead de cabeceras repetidas** — 1500 bytes pueden terminar en 185 fragmentos y pasar de 1500 a 5180 bytes en el cable; (ii) **unidad mínima de 8 bytes**; (iii) **la pérdida de un solo fragmento descarta el datagrama entero**, con probabilidad de éxito `(1-p)^n` que decae exponencialmente; (iv) **el reensamblaje exige estado y temporizador** en el destino, y si vence se pierde todo; (v) **el solapamiento es un agujero de seguridad**, porque el segundo fragmento sobrescribe al primero (`RFC 5722` lo prohíbe en IPv6); (vi) **cada fragmento es un paquete IP independiente para los filtros**, lo que permite **evadir firewalls** que solo inspeccionan el primero.

3.3 Repaso: `2000-07` Problema 2

> **Fuente:** Examen `sirc0007`, Problema 2 — protocolo fiable sobre 3 subredes con MTU distinta.

Es la **misma idea con transporte confiable**: entre dos extremos unidos por subredes con MTUs distintas no se puede "confiar en que TCP fragmenta". Si la capa 3 no fragmenta (o si el camino es IPv6), **el transporte tiene que fragmentar él mismo**, en el emisor, y desensamblar en el receptor: el mismo

esquema de §1.5, pero el que decide los tamaños es el emisor del segmento de transporte y no un router. Menciónalo si te lo preguntan como ejemplo de por qué el transporte tiene su propio control de tamaño.

PARTE 4 — CIERRE

📌 Resumen

1. **MTU** = la mayor PDU de capa 3 que el enlace transmite sin fragmentar. En Ethernet son **1500 bytes**; la trama llega a 1518 con cabecera y FCS.
2. Los **3 campos** de la fragmentación IPv4 son **Identification**, **Flags (con `DF`)** y **Fragment Offset**.
3. El `Fragment Offset` **va en unidades de 8 bytes**, y salvo el último el payload debe ser múltiplo de 8.
4. El router **descarta y avisa** si `DF = 1` (`ICMP tipo 3 código 4`); **fragmenta** si `DF = 0`. Nunca fragmenta algo que entra entero.
5. El ICMP tipo 3 código 4 existe para que el emisor **no repita el error** y **baje su tamaño**; lleva la MTU del enlace bloqueante en los **bytes 6-7**.
6. Los problemas: **overhead de cabeceras**, **unidad de 8 bytes**, **temporizador de reensamblaje**, **solapamiento (agujero de seguridad)**, **perder un fragmento = perder el datagrama entero**, y **evasión de filtros**.
7. El **Path MTU** es el **mínimo** de los MTUs del camino; **varía con el tiempo** y **es distinto en cada sentido** si el ruteo es asimétrico.
8. El **PMTUD (`RFC 1191`)** = `DF = 1`, recibir el ICMP, **bajar y cachear** el PMTU. La MTU exacta viene en los bytes 6-7; usarla da resultado **exacto**, mientras que multiplicar por 0.85 **siempre subestima**.
9. El **reensamblaje** necesita índice por offset, **detección de solapamiento**, **temporizador** y la clave `(IP destino, ID, protocolo)`. **No basta con ver `MF = 0`.**
10. El **PMTUD black hole** produce el síntoma **"SSH funciona, HTTPS se cuelga"**, y se diagnostica con `ping -M do -s <tamaño>`.

✅ Autoevaluación

1. Un datagrama IPv4 de 1500 bytes con `IHL = 5` pasa por un enlace de MTU 576. ¿Cuántos fragmentos y cuál es el `Fragment Offset` del último?

▶ Respuesta

2. ¿Por qué el router **descarta** un datagrama con `DF = 1` que no entra, en vez de fragmentarlo igual?

▶ Respuesta

3. ¿Qué pasa si se pierde **uno solo** de los fragmentos de un datagrama partido en 5? Explicá con un número.

▶ Respuesta

4. ¿Por qué el `Fragment Offset` está en unidades de 8 bytes y no de 1? ¿Qué consecuencia práctica tiene?

▶ Respuesta

5. Un `traceroute` muestra un salto con `*`. ¿Podés concluir que ese router está caído?

▶ Respuesta

6. ¿Por qué la solución oficial multiplica por 0.85, y por qué está mal?

▶ Respuesta

7. ¿Cuál es el MTU mínimo absoluto en IPv6 y de dónde sale? ¿Y en IPv4?

▶ Respuesta

8. ¿Por qué el jumbo frame de 9000 es "una convención y no un estándar IEEE"? ¿Y por qué no se usa en Internet?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1012` / `sirc1012`** — Problema 2 completo (30 pts: MTU, fragmentación, Path MTU e **implementación del descubrimiento**); incluye la tabla de tipos y códigos de ICMP
- **Examen `eirc0612` / `sirc0612`** — P3 (10 pts: por qué fragmentar, cómo funciona, problemas asociados y **algoritmo de reensamblaje**)
- **Examen `sirc0007`** — Problema 2: protocolo fiable sobre 3 subredes con MTU distinta
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 §5.5.2 "Fragmentación"
- **RFC 791** — Internet Protocol (campos de fragmentación, MTU mínimo 68)
- **RFC 1122 §3.2.1** — MTU mínimo de 576 bytes
- **RFC 1191** — Path MTU Discovery (1990): la MTU del enlace bloqueante en los bytes 6-7 del ICMP
- **RFC 8200 §5** — IPv6 (2017): **MTU mínimo absoluto de 1280**
- **RFC 8201** — Path MTU Discovery for IPv6 (2017)
- **RFC 8899** — DPLPMTUD (*Standards Track*, sep-2020; sucesor de RFC 4821) · **RFC 4821** — *Experimental*, 2007
- **RFC 5722** (2009) — Prohibición de fragmentos solapados en IPv6 · **RFC 6946** (2013) — fragmentos atómicos
- **RFC 9002 §6** — QUIC: PLPMTUD integrado, sin dependencia de ICMP
- **Clase siguiente:** 36 — IPv6